Metodologia de Projetos de TI	
Mínimo para o 7,0	M1. Elaboração do Termo de Abertura e Encerramento do Projeto M2. Utilização de ferramentas de gerenciamento de projetos, como jira, miro etc. No planejamento do projeto. M3. Definição do cronograma do projeto, utilizando ferramentas de planejamento de projetos
Extra	E1. Definição do planejamento e alocação de todos os recursos para o projeto. E2. Alinhamento do projeto com o negócio (empresas do grupo).

2º Ano

Banco de Dados 2	
Mínimo para o 7,0	- Usar, obrigatoriamente, os dois bancos de dados: • MongoDB • REDIS ***OBS:*** O MongoDB deve ser usado em alguma interação conversacional. O Redis deve ser usado para fila de processamento e/ou ranking em tempo real.
Extra	**Usar o Neo4J:** Você deve modelar as entidades principais do projeto como grafo e responder pelo menos uma pergunta de negócio que exija **traversal** (é esse nome mesmo, isso não uma simples busca por nó)
Engenharia de Software	
Mínimo para o 7,0	M1. Criar um mecanismo de acompanhamento (planejamento) do projeto via JIRA, TRELLO ou outro semelhante e me incluir; M2. Elaborar um documento que apresente os requisitos funcionais do sistema; M3. Elaborar o diagrama de casos de uso do sistema. Colocar todos os casos de uso e relacioná-los com os requisitos funcionais apresentados M4. Escolha 2 processos no seu sistema e elabore um diagrama de atividades para cada um deles. M5. Elaborar um documento em que esteja indicada a dinâmica de trabalho da equipe e a pessoa responsável por back, front, banco de dados etc.
Extra	E1. Usar algum padrão de projeto, indicar em que parte do código foi usado e com que finalidade. E2. Utilizar um padrão de testes (como TDD) e indicar onde será utilizado e como.
Desenvolvimento de Aplicativos Móveis	
Mínimo para o 7,0	- Controle de Acesso

	- App deve utilizar um recurso do hardware do dispositivo (câmera / telefone / sms) - Integrar a utilização de API (consumir) - Persistência de Dados através do Firebase ou SQLite (livre escolha) - Chatbot operacional
Extra	- Configurar Perfil (Avatar) - Uso de Notificação para o App - App ajustado para apresentação no Celular e Tablet. - Utilizar o GPS do dispositivo no projeto
Desenvolvimento 2	
Mínimo para o 7,0	- Criar uma API REST em Java usando o Spring MVC para acesso a um banco de dados SQL Postgres; - Utilizar o Spring Data JPA para integrar a aplicação Java com o banco de dados Postgres; - Implementar os métodos do CRUD no banco de dados por meio da API; - Aplicar a divisão de responsabilidades criando os pacotes que implementam o padrão MVC; - Validar as entradas recebidas pela API para garantir a consistência e integridade dos dados; - Implementar tratamento de exceções centralizado para lidar com erros de validação e de banco de dados, entre outros, retornando respostas HTTP apropriadas com informações úteis para o cliente.
Extra	- Utilizar o Swagger para gerar documentação automática da API, descrevendo os endpoints disponíveis, seus parâmetros e respostas esperadas; - Acionar as Procedures e Functions criadas na disciplina Modelagem de Dados; - Implementar autenticação e autorização para proteger os endpoints da API usando o Spring Security; - Criar mais uma API REST em Java usando o Spring MVC para acesso a um banco de dados NoSQL.
Desenvolvimento de Aplicações Dinâmicas	
Mínimo para o 7,0	- Projeto criado com Vite e template React TypeScript — todos os arquivos dentro de src/ com extensão .tsx ou .ts; proibido .js ou .jsx dentro de src/; - Componentes em src/components/ e páginas em src/pages/, cada um em pasta própria com index.tsx — convenção aplicada consistentemente em todo o projeto; - Mínimo de 6 componentes com responsabilidade única — nenhum componente mistura renderização, lógica de negócio ou acesso a src/services/;

	- Interfaces TypeScript para props de todos os componentes e entidades do domínio em src/types/, organizadas por arquivo de domínio — proibido any em src/; - Toda comunicação com API externa isolada em src/services/, com arquivos por recurso, retorno tipado Promise<Tipo> e try/catch — proibido fetch em componentes ou páginas; chaves e URLs privadas em variáveis de ambiente, nunca versionadas; - Estado local gerenciado com useState, campos relacionados agrupados em um único objeto — proibida mutação direta de arrays e objetos; - useEffect com array de dependências explícito e correto — o avaliador verificará removendo uma dependência ao vivo e observando se o comportamento muda; - Listas dinâmicas renderizadas com key de identificador único do dado — proibido key={index} em listas com adição, remoção ou reordenação; - Parâmetros de rota recuperados com useParams com verificação de presença antes do uso — proibido acessar parâmetro sem tratar undefined; - Mínimo de três rotas com React Router, navegação via <Link> ou useNavigate — proibido window.location.href; rota curinga path="*" com página de erro e botão de retorno; - Acessibilidade mínima seguindo WCAG 2.1 nível AA: <button> para todas as ações — proibido <div onClick>; <label> associado a cada <input> via htmlFor; imagens com alt descritivo ou alt="" quando decorativas; contraste mínimo de 4,5:1 para texto normal e 3:1 para texto grande; erros no DOM com role="alert"; aplicação integralmente operável por teclado com foco visível em todos os elementos interativos — testado ao vivo sem mouse na apresentação; - Feedback visual para operações assíncronas com três estados no DOM: carregando (spinner, skeleton ou aria-live), sucesso e erro — proibido alert() e console.log como único tratamento; - Aplicação publicada em Vercel ou GitHub Pages com link no README.md — acessada ao vivo na apresentação; - Mínimo de 20 commits no padrão Conventional Commits (feat:, fix:, refactor:, docs:) cobrindo ao menos 4 semanas de desenvolvimento — proibidos commits genéricos.
Extra	- Context API com Provider dedicado, hook customizado de consumo e tipagem completa — grupo demonstra oralmente o problema de prop drilling resolvido, apontando no código; - useReducer com State e Action tipados em union types — grupo demonstra oralmente por que não múltiplos useState, apontando a complexidade real do estado;

	- Hook customizado com retorno tipado, estado de loading, erro tratado e cancelamento via AbortController para evitar atualização de estado em componente desmontado; - React.lazy e Suspense por página — demonstrado ao vivo na aba Network do DevTools no carregamento inicial de cada rota; - localStorage com schema versionado — campo _versao no objeto salvo, verificação ao carregar e migração ou descarte seguro em caso de incompatibilidade de versão; - useCallback e useMemo com justificativa de performance — avaliador comenta a otimização ao vivo e grupo demonstra a diferença no React DevTools Profiler; - Proteção contra XSS: proibido dangerouslySetInnerHTML com dados externos; todo conteúdo dinâmico via interpolação JSX {variavel} — grupo demonstra no código onde dados externos são tratados antes de renderizados; - Validação e sanitização de formulários: campos com tipo, tamanho máximo e mensagem de erro específica por campo em src/utils/ — nenhum dado enviado ao serviço sem passar por função de validação tipada; - Rotas privadas com componente PrivateRoute que redireciona para login quando não autenticado — estado de autenticação via Context, nunca verificado diretamente no componente de página; - Acessibilidade avançada seguindo WCAG 2.1 nível AA completo: componentes interativos complexos (modais, menus, abas) implementados com padrões ARIA corretos (aria-expanded, aria-controls, aria-haspopup, aria-selected) e gerenciamento de foco programático — foco movido para o elemento correto ao abrir e devolvido ao elemento de origem ao fechar; landmarks semânticos (<header>, <main>, <nav>, <footer>) estruturando todas as páginas; grupo demonstra ao vivo navegação com leitor de tela (NVDA ou VoiceOver) sem perda de contexto ou armadilha de foco.
Desenvolvimento de Operações Ágeis	
Mínimo para o 7,0	M1. Aplicar os recursos de CodeReview e Pull Request juntamente com as aprovações. M2. Utilizar a Cloud como infraestrutura para hospedar o projeto. M3. Utilizar tecnologia de container e orquestração (Ex. Docker e Kubernetes)
Extra	E1. Criar o README do projeto - completo E2. Definir Template de PullRequest e aplicar durante o desenvolvimento E3. Elaborar Pipeline para implantação (Ex. Github Actions ou Jenkins) **Entrega:** Criar um documento de entregas, descrevendo cada requisito e indicando o link em que se encontra, de maneira detalhada, de modo que, ao clicar, leve diretamente para o requisito especificado.

Projeto Interdisciplinar

Solicitações por Disciplina | 14/05/2026

Experiência do Usuário - UX	
Mínimo para o 7,0	1. Elaboração de uma apresentação contendo o processo de ideação do produto: Dados levantados através das pesquisas qualitativas e quantitativas. Matriz CSD Análise de Benchmark Desk research Desenho da persona e mapa de empatia Jornada do usuário Fluxo/Site map 2. Protótipo mobile em alta fidelidade 3. Análise Heurística 4. Compilado de testes com usuários (tempo da tarefa, dificuldade da tarefa) 5. Teste A/B Matriz de priorização das atividades pós testes de acordo com grau de severidade 6. Mapa de Features (core, adjacente, transformacional) 7. Matriz de Proposta de valor
Extra	V2 do protótipo com alterações realizadas e nova documentação de testes. Jobs to be done, comparação da jornada de usuário ideal e relatório de melhorias. UI kit - documentação do design system
Modelagem de Dados	
Mínimo para o 7,0	**1. Modelagem de Dados** • **Novo Banco Normalizado:** Criar um novo banco de dados relacional (PostgreSQL) normalizado até a **3ª Forma Normal (3FN)**, caso seja necessário, até a **5ª Forma Normal**. • **Relacionamentos:** Definir claramente todas as PKs, FKs e cardinalidades (1:1, 1:N, N:N). • **Massa de Dados:** Carga inicial de pelo menos **500 registros** verossímeis para testes de volume. **2. Engenharia de Dados e ETL** • **Integração RPA:** Orquestrar a movimentação de dados entre o banco legado e o novo banco normalizado via RPA, garantindo a integridade e tratando falhas na carga. • **Camada de BI (Data Mart):** Criar **Views Analíticas** desenhadas para ferramentas de BI, utilizando a técnica de **Modelagem Dimensional (Star Schema ou Snowflake)**. • **Transformação (ETL no SQL):** ○ **CTEs:** Utilizar para organizar a lógica de transformação de dados. ○ **Window Functions:** Implementar cálculos analíticos (ex: *Running Total*, *Ranking*) nas views de BI.

	3. Objetos Lógicos e Otimização • **SQL Avançado:** Mínimo de 2 Functions e 2 Procedures para regras de negócio críticas. • **Triggers de Auditoria:** Logs completos (NEW, OLD, TG_OP, CURRENT_USER) em pelo menos 2 tabelas. • **Otimização:** Analisar consultas lentas com EXPLAIN ANALYZE e aplicar **Índices** (SQL) para otimização.
Extra	**4. Governança de Dados:** • **Catálogo de Dados:** Criar uma tabela técnica de metadados descrevendo cada tabela, suas colunas, regras de negócio e níveis de acesso. • **Segurança e Integridade:** Definir procedimentos documentados para **Backup e Recuperação** de falhas. **5. Monitoramento:** Automatizar o registro de acessos **DAU (Daily Active User)** diretamente no banco Postgres. **6. Estruturas Avançadas:** Implementar, onde fizer sentido, **CTE Recursiva** ou **Herança de Tabelas**.
Business Intelligence	
Mínimo para o 7,0	**Visualização** - Objetivo: Apresentar informações coletadas a partir do uso da aplicação de forma organizada em um dashboard; Requisitos: • Conectividade com Dados: A ferramenta deve estar conectada a fonte de dados que sua aplicação gera. Visualizações Necessárias: • Gráficos de Barra e Linha: Para mostrar a evolução e comparações de dados coletados. • Cards com os KPIs e desvios quando necessário; • Histogramas e boxplots para visualização das distribuições • Mapas (se aplicável): Caso a pesquisa envolva dados geográficos. • Filtros Interativos: Permitir que o usuário filtre os dados por diferentes critérios (ex.: período, grupo etc.). **Relatório Gerencial:** • Apresentação da base; • Problema analisado; • Estratégias de tratamento dos dados; • KPIs;
Extra	**Job e Pipelines:** • Criar um pipeline de dados no Databricks com leitura, tratamento, transformação, cálculo de KPIs e salvamento da base final. • O pipeline deve usar a fonte de dados gerada pela aplicação e produzir uma base tratada para o dashboard. • Criar um job no Databricks para executar o pipeline sem rodar manualmente cada célula.

	• O job pode ter execução manual ou agendada, conforme a proposta do projeto. • O dashboard deve ser construído a partir da base final gerada pelo pipeline. • Apresentar evidências do job: configuração, histórico de execução, status e resultado gerado. • O relatório gerencial deve explicar o fluxo do pipeline, as etapas automatizadas e a relação com os KPIs e o dashboard.
Inteligência Artificial	
Mínimo para o 7,0	Desenvolver uma API usando fast API (ou Flask) para um sistema baseado em IA que use modelos (Gemini, GPT, DeepSeek, Claude, Groq, etc). *Importante: a escola **NÃO** arcará com nenhuma despesa de API de IA generativa*. • O grupo deverá definir o escopo do sistema. • O sistema deve ser multiagente e deve ter, no mínimo 5 agentes. • Uso de Langchain para criação dos agentes. • Uso de Langgraph para orquestração de agentes. • Controle as sessões por usuário. • Memória de longo prazo. • MCP, A2A – Integrações entre sistemas e/ou agentes externos. • Mitigação de alucinações com uso de RAG, juízes e guardrail: • Agente com RAG: Indicação das fontes externas usadas para aumentar a geração da resposta da IA (arquivos locais, consulta à internet...) - No mínimo uma fonte externa deve ser consumida. • Agente de juiz para controle de alucinações. • Implementação de guardrail. • Precisa ter observabilidade/SRE do sistema com: - Custo estimado para cenários de 100 e 1000 usuários semanais - Latência interagentes e tempo total de resposta - Indice de erros - Custo / Retorno (ROI) - Custo por resolução Precisa de um desenho de arquitetura de alto nível.
Extra	Complexidade do projeto.